```
import numpy as np
import scipy.special as special

class CuckooSearchDronePath:
    def __init__(self, start, goal, obstacles, field_bounds, num_waypoints=10, num_nests=25, pa=0.25, max_iter=100):
        self.start = np.array(start)
        self.goal = np.array(goal)
        self.obstacles = obstacles  # List of tuples: (x, y, radius)
        self.bounds = field_bounds  # Tuple: (xmin, xmax, ymin, ymax)
        self.num_waypoints = num_waypoints  # Intermediate waypoints (excluding start/goal)
        self.num_nests = num_nests
        self.pa = pa
        self.max_iter = max_iter

        # Dimensions per solution: num_waypoints * 2 (x and y coordinates)
        self.dim = self.num_waypoints * 2

        # Define Levy Flight parameters
        self.beta = 1.5
        sigma_num = special.gamma(1 + self.beta) * np.sin(np.pi * self.beta / 2)
        sigma_den = special.gamma((1 + self.beta) / 2) * self.beta * (2 ** ((self.beta - 1) / 2))
        self.sigma = (sigma_num / sigma_den) ** (1 / self.beta)

    def reconstruct_path(self, decision_variables):
        """Converts the flat optimization array into a full continuous path matrix."""
        waypoints = decision_variables.reshape((self.num_waypoints, 2))
        return np.vstack([self.start, waypoints, self.goal])

    def calculate_fitness(self, decision_variables):
        """Evaluates the path based on distance, smoothness, and obstacle collisions."""
        path = self.reconstruct_path(decision_variables)

        # 1. Total Distance Cost
        distances = np.linalg.norm(np.diff(path, axis=0), axis=1)
        total_distance = np.sum(distances)

        # 2. Smoothness Cost (Penalize sharp turning angles)
        smoothness_cost = 0.0
        for i in range(1, len(path) - 1):
            v1 = path[i] - path[i-1]
            v2 = path[i+1] - path[i]
            norm1, norm2 = np.linalg.norm(v1), np.linalg.norm(v2)
            if norm1 > 1e-5 and norm2 > 1e-5:
                cos_theta = np.dot(v1, v2) / (norm1 * norm2)
                cos_theta = np.clip(cos_theta, -1.0, 1.0)
                angle = np.arccos(cos_theta)
                smoothness_cost += angle ** 2

        # 3. Obstacle Penalty (Check close proximity along the path segments)
        obstacle_penalty = 0.0
        for i in range(len(path) - 1):
            p1, p2 = path[i], path[i+1]
            for ox, oy, r in self.obstacles:
                # Calculate minimum distance from obstacle center to path segment p1->p2
                obs_center = np.array([ox, oy])
                line_vec = p2 - p1
                line_len = np.linalg.norm(line_vec)
                if line_len > 1e-5:
                    line_unit = line_vec / line_len
                    proj = np.dot(obs_center - p1, line_unit)
                    proj = np.clip(proj, 0, line_len)
                    closest_point = p1 + proj * line_unit
                    dist_to_obs = np.linalg.norm(closest_point - obs_center)

                    if dist_to_obs < r:
                        obstacle_penalty += (r - dist_to_obs) * 5000.0  # Massive penalty factor

        return total_distance + (10.0 * smoothness_cost) + obstacle_penalty

    def apply_boundaries(self, nests):
        """Enforces field boundary constraints on the flight coordinates."""
        xmin, xmax, ymin, ymax = self.bounds
        for i in range(self.num_nests):
            nests[i, 0::2] = np.clip(nests[i, 0::2], xmin, xmax)  # x-coordinates
            nests[i, 1::2] = np.clip(nests[i, 1::2], ymin, ymax)  # y-coordinates
        return nests

    def levy_flight(self, current_nest, best_nest):
        """Generates random steps using a heavy-tailed Levy distribution."""
        u = np.random.normal(0, self.sigma, size=self.dim)
        v = np.random.normal(0, 1, size=self.dim)
```

```
        v = np.random.normal(0, 1, size=self.dim)
        step = u / (np.abs(v) ** (1 / self.beta))

        # Step scaling proportional to distance from global best solution
        stepsize = 0.01 * step * (current_nest - best_nest)
        return current_nest + stepsize * np.random.normal(size=self.dim)

    def optimize(self):
        xmin, xmax, ymin, ymax = self.bounds

        # Initialize nests randomly between the bounds
        nests = np.zeros((self.num_nests, self.dim))
        nests[:, 0::2] = np.random.uniform(xmin, xmax, (self.num_nests, self.num_waypoints))
        nests[:, 1::2] = np.random.uniform(ymin, ymax, (self.num_nests, self.num_waypoints))
        nests = self.apply_boundaries(nests)

        # Evaluate initial fitness
        fitness = np.array([self.calculate_fitness(nest) for nest in nests])
        best_idx = np.argmin(fitness)
        best_nest = nests[best_idx].copy()
        best_fitness = fitness[best_idx]

        for iteration in range(self.max_iter):
            # 1. Levy Flights to generate new paths
            new_nests = np.zeros_like(nests)
            for i in range(self.num_nests):
                new_nests[i] = self.levy_flight(nests[i], best_nest)

            new_nests = self.apply_boundaries(new_nests)

            # Select random nests to compare against
            for i in range(self.num_nests):
                f_new = self.calculate_fitness(new_nests[i])
                random_partner = np.random.randint(0, self.num_nests)

                if f_new < fitness[random_partner]:
                    nests[random_partner] = new_nests[i]
                    fitness[random_partner] = f_new

            # 2. Abandon worst nests with probability Pa
            sort_idx = np.argsort(fitness)
            nests = nests[sort_idx]
            fitness = fitness[sort_idx]

            num_abandoned = int(self.pa * self.num_nests)
            keep_idx = self.num_nests - num_abandoned

            # Biased random walk for building new nests
            for i in range(keep_idx, self.num_nests):
                r1, r2 = np.random.randint(0, keep_idx, size=2)
                stepsize = np.random.rand() * (nests[r1] - nests[r2])
                nests[i] = nests[i] + stepsize

            nests = self.apply_boundaries(nests)
            fitness = np.array([self.calculate_fitness(nest) for nest in nests])

            # 3. Update global best path
            current_best_idx = np.argmin(fitness)
            if fitness[current_best_idx] < best_fitness:
                best_fitness = fitness[current_best_idx]
                best_nest = nests[current_best_idx].copy()

            print(f"Iteration {iteration+1}/{self.max_iter} - Best Cost: {best_fitness:.2f}")

        return self.reconstruct_path(best_nest)

# Example Execution
if __name__ == "__main__":
    # Define agricultural layout
    start_pos = [0.0, 0.0]
    end_pos = [100.0, 100.0]
    field = (0.0, 100.0, 0.0, 100.0)  # xmin, xmax, ymin, ymax

    # Obstacles: (x_center, y_center, radius) e.g., barns, power poles, silos
    obstacles_list = [
        (30.0, 40.0, 10.0),
        (50.0, 50.0, 12.0),
        (75.0, 60.0, 8.0)
    ]

    cuckoo_planner = CuckooSearchDronePath(
        start=start_pos,
        goal=end_pos,
```

```
        obstacles=obstacles_list,
        field_bounds=field,
        num_waypoints=8,    # Higher values allow complex maneuvers but take longer
        num_nests=30,
        pa=0.25,
        max_iter=50
    )

    optimized_trajectory = cuckoo_planner.optimize()
    print("\nOptimal Path Coordinates:")
    print(optimized_trajectory)
```

```
Iteration 5/50 - Best Cost: 26312.25
Iteration 6/50 - Best Cost: 11812.10
Iteration 7/50 - Best Cost: 11812.10
Iteration 8/50 - Best Cost: 11812.10
Iteration 9/50 - Best Cost: 11812.10
Iteration 10/50 - Best Cost: 11812.10
Iteration 11/50 - Best Cost: 11812.10
Iteration 12/50 - Best Cost: 11812.10
Iteration 13/50 - Best Cost: 11812.10
Iteration 14/50 - Best Cost: 11812.10
Iteration 15/50 - Best Cost: 11812.10
Iteration 16/50 - Best Cost: 11812.10
Iteration 17/50 - Best Cost: 11812.10
Iteration 18/50 - Best Cost: 11812.10
Iteration 19/50 - Best Cost: 11812.10
Iteration 20/50 - Best Cost: 11812.10
Iteration 21/50 - Best Cost: 11812.10
Iteration 22/50 - Best Cost: 11812.10
Iteration 23/50 - Best Cost: 11812.10
Iteration 24/50 - Best Cost: 11812.10
Iteration 25/50 - Best Cost: 11812.10
Iteration 26/50 - Best Cost: 11812.10
Iteration 27/50 - Best Cost: 11812.10
Iteration 28/50 - Best Cost: 11812.10
Iteration 29/50 - Best Cost: 11812.10
Iteration 30/50 - Best Cost: 11812.10
Iteration 31/50 - Best Cost: 11812.10
Iteration 32/50 - Best Cost: 11812.10
Iteration 33/50 - Best Cost: 11812.10
Iteration 34/50 - Best Cost: 11812.10
Iteration 35/50 - Best Cost: 11812.10
Iteration 36/50 - Best Cost: 11812.10
Iteration 37/50 - Best Cost: 11812.10
Iteration 38/50 - Best Cost: 11812.10
Iteration 39/50 - Best Cost: 11812.10
Iteration 40/50 - Best Cost: 11812.10
Iteration 41/50 - Best Cost: 11812.10
Iteration 42/50 - Best Cost: 11812.10
Iteration 43/50 - Best Cost: 11812.10
Iteration 44/50 - Best Cost: 11812.10
Iteration 45/50 - Best Cost: 11812.10
Iteration 46/50 - Best Cost: 11812.10
Iteration 47/50 - Best Cost: 11812.10
Iteration 48/50 - Best Cost: 11812.10
Iteration 49/50 - Best Cost: 11812.10
Iteration 50/50 - Best Cost: 11812.10

Optimal Path Coordinates:
[[  0.           0.        ]
 [ 60.45864739  10.42938157]
 [ 96.02053819  82.21724129]
 [ 85.50973027  34.05434306]
 [ 12.54960378  18.7129023 ]
 [ 79.85405292  10.27811575]
 [ 13.03686962  37.86023388]
 [ 68.65713061  85.34930552]
 [ 44.83492579  88.16499998]
 [100.         100.        ]]
```